Project Title

AI System to Automatically Review and Summarize Research Papers

Internship Program: Infosys Springboard Internship
Intern Name: Tanuj Kumar
Mentor: Ms. M. Likhita

Project Overview

Objective

The objective of this project is to design and implement an AI-powered system that automates the process of reviewing, comparing, and summarizing academic research papers.

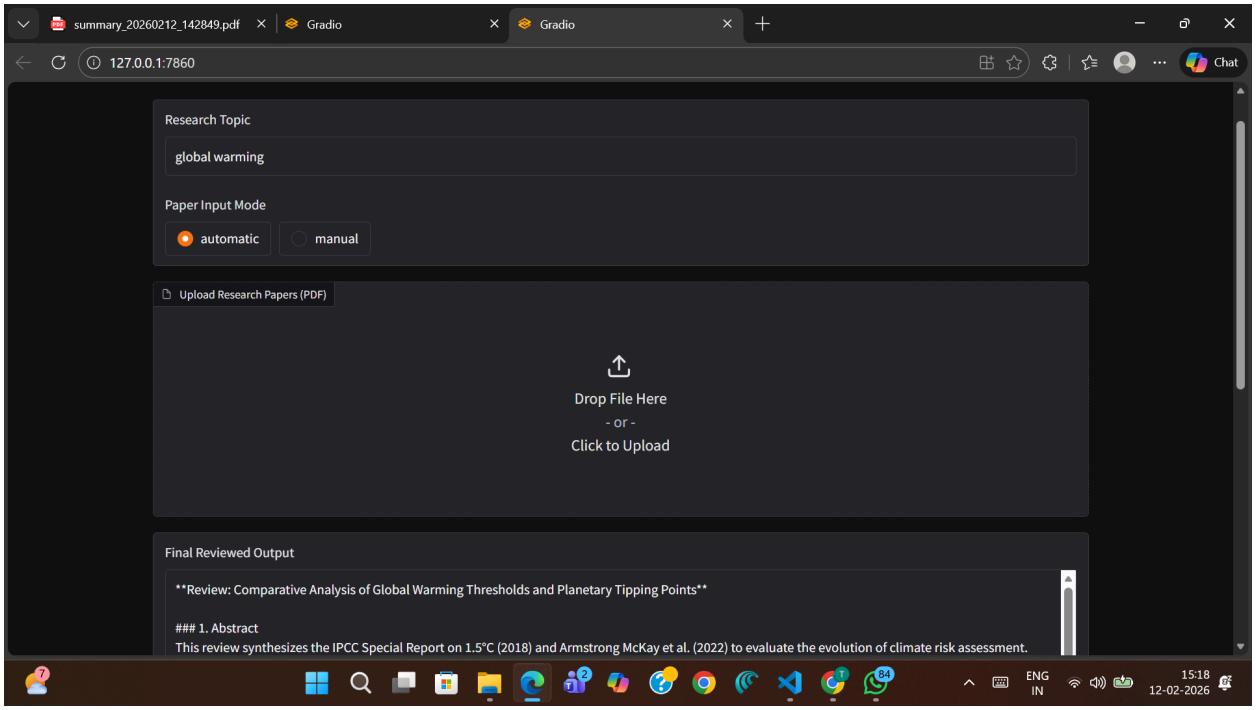
The system reduces manual effort in literature review by:

- Automatically retrieving research papers
- Extracting PDF content
- Performing section-wise comparison
- Generating structured summaries
- Producing final refined academic reports

The system supports two modes:

- **Automatic Mode:** Fetch research papers using Semantic Scholar API
- **Manual Mode:** User uploads or places PDFs in local directory

The current phase focuses on building a complete end-to-end AI research review pipeline.



Task Breakdown (As Assigned by Mentor)

The following tasks were defined and implemented:

1. Environment Setup
2. Automated Paper Retrieval
3. PDF Download and Storage
4. Text Extraction
5. Section-wise Paper Analysis
6. AI-based Draft Generation
7. Review & Refinement
8. Output Storage (TXT + PDF)

Environment Setup

The development environment was configured using:

- Python 3.x

- Virtual Environment (venv)
- Visual Studio Code

All required dependencies were installed using a requirements.txt file.

Libraries Used

- requests – for Semantic Scholar API
- google-generativeai / Gemini API – for LLM-based analysis
- langchain – prompt orchestration
- gradio – user interface
- pymupdf4llm – PDF text extraction
- LangGraph - for project worlflow/control
- pandas – dataset handling
- os, datetime – file management
- reportlab – PDF generation

This ensured stable execution of the automated research workflow.

System Architecture

The system follows a modular pipeline architecture:

User (Gradio UI)
↓
app.py (Interface Layer)
↓
main.py (Pipeline Controller)
↓
workflow.py
↓
Planner → Paper Retrieval → PDF Downloader
→ Text Extractor → Analyzer → Draft Generator → Reviewer
↓
Final Output (TXT + PDF)

This architecture ensures:

- Clear separation of responsibilities
- Scalability

- Easy debugging
- Maintainability

Automated Paper Search

The system uses the **Semantic Scholar Graph API** to retrieve relevant research papers based on a user-provided topic.

Retrieved Metadata Includes:

- Paper title
- Authors
- Year
- Open-access PDF link

The system filters papers based on:

- Open-access PDF availability
- Valid download link

Only papers with accessible PDFs are selected.

Error handling ensures:

- API failure detection
- Missing PDF handling
- Retry-safe execution

Paper Selection and PDF Download

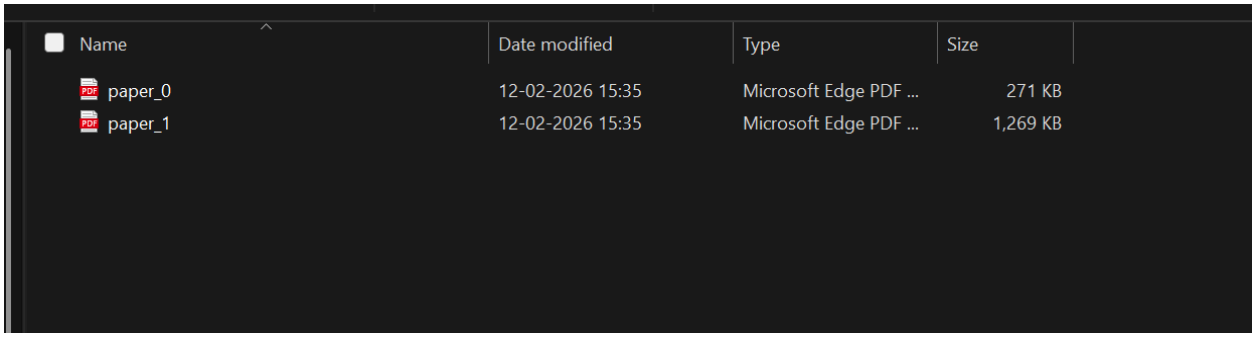
From retrieved results:

- Only open-access papers are downloaded
- Files are stored in topic-specific folders
- Sequential numbering is applied

- Timestamp-based organization prevents overwrite

Example:

data/papers/



Console logs show download progress.

This ensures reproducibility and organized dataset creation.

Text Extraction

The system extracts full PDF text using **PyMuPDF4LLM**.

Features:

- Multi-page support
- Clean text extraction
- Handles formatting inconsistencies

Extracted text is prepared for AI-based comparison.

Section-wise Analysis

The Analyzer module performs:

- Objective comparison
- Methodology comparison
- Results comparison

- Identification of similarities
- Extraction of key insights

The system supports comparison of up to **3 research papers** simultaneously.

Gemini LLM performs semantic reasoning.

Draft Generation

The Draft Generator creates a structured academic output including:

- Research Topic
- Input Mode
- Generated Timestamp
- Abstract (≤ 100 words)
- Methodology
- Results
- Key Insights
- Limitations

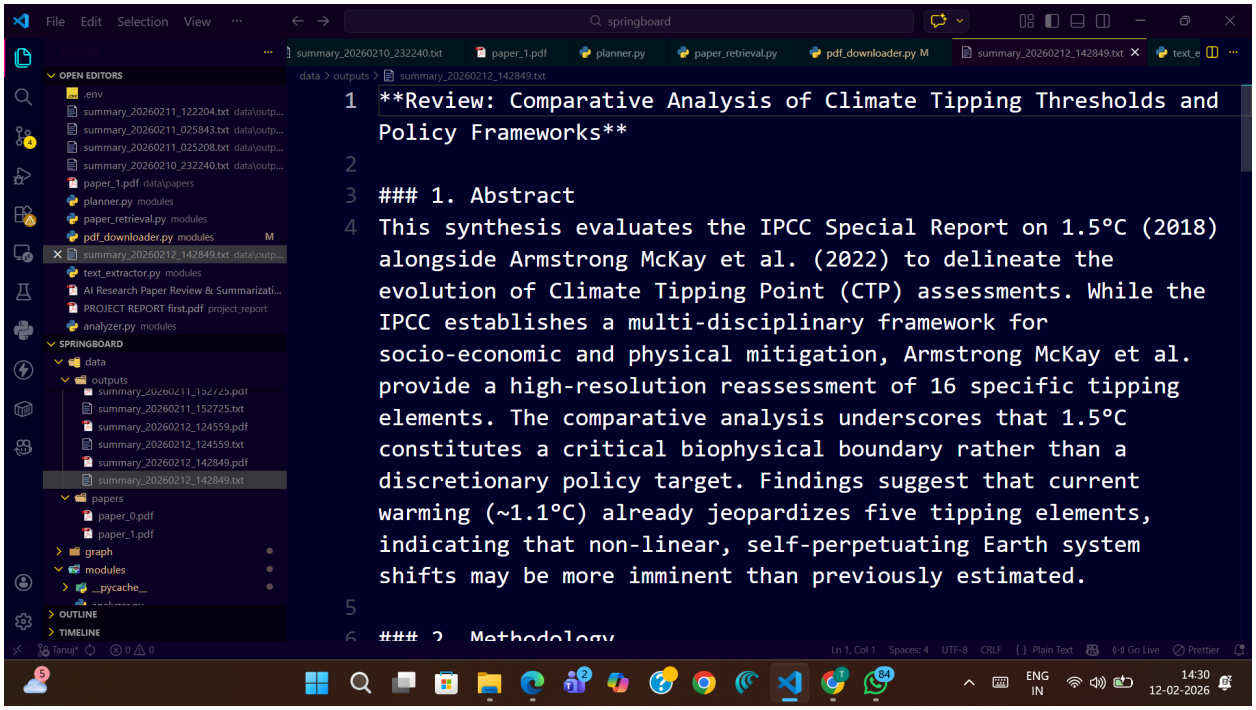
This ensures structured academic format.

Review and Refinement

The Reviewer module:

- Improves academic tone
- Removes redundancy
- Enhances clarity
- Preserves structure

Unwanted LLM commentary is filtered out.



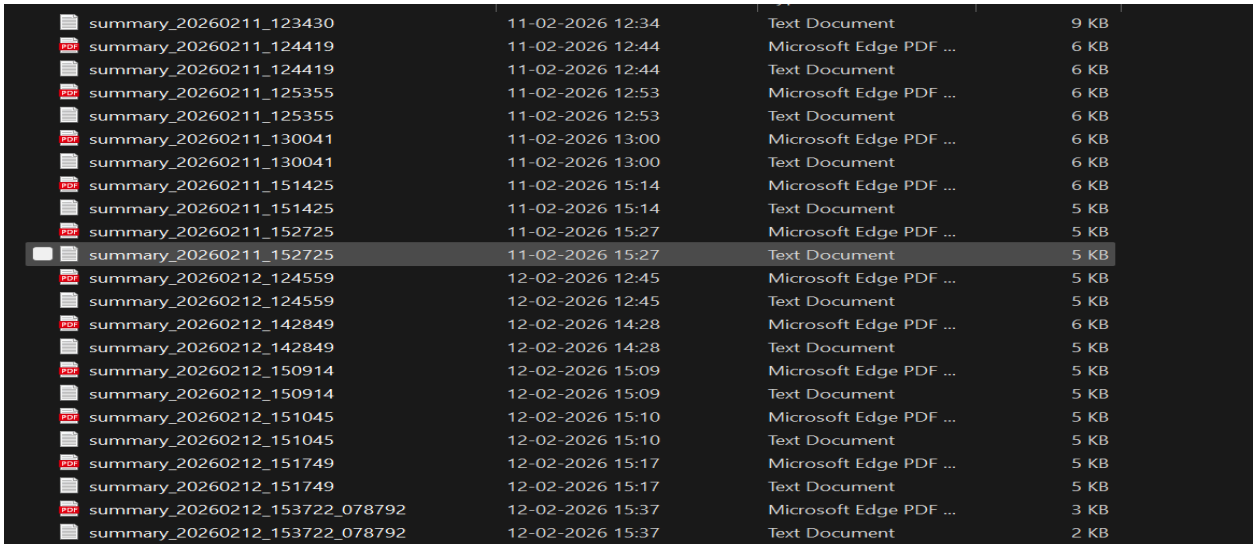
Output Storage

Final output is stored in:

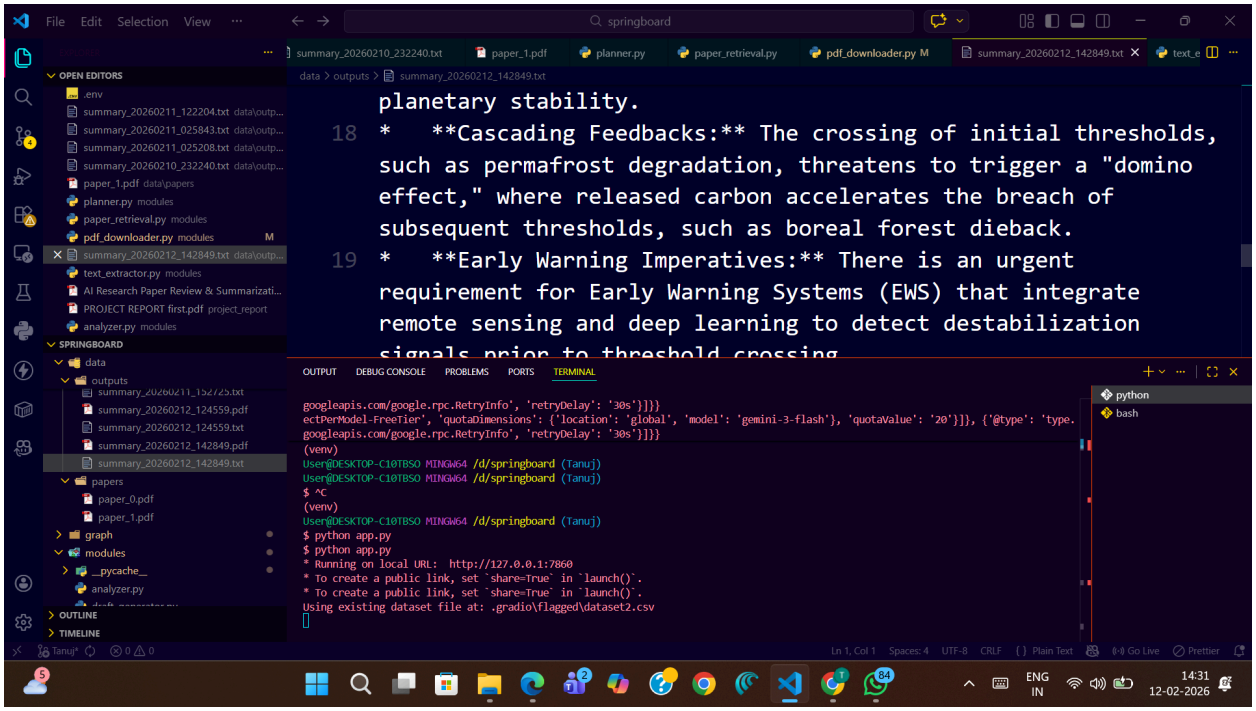
data/outputs/

Files generated:

- summary_timestamp.txt
- summary_timestamp.pdf



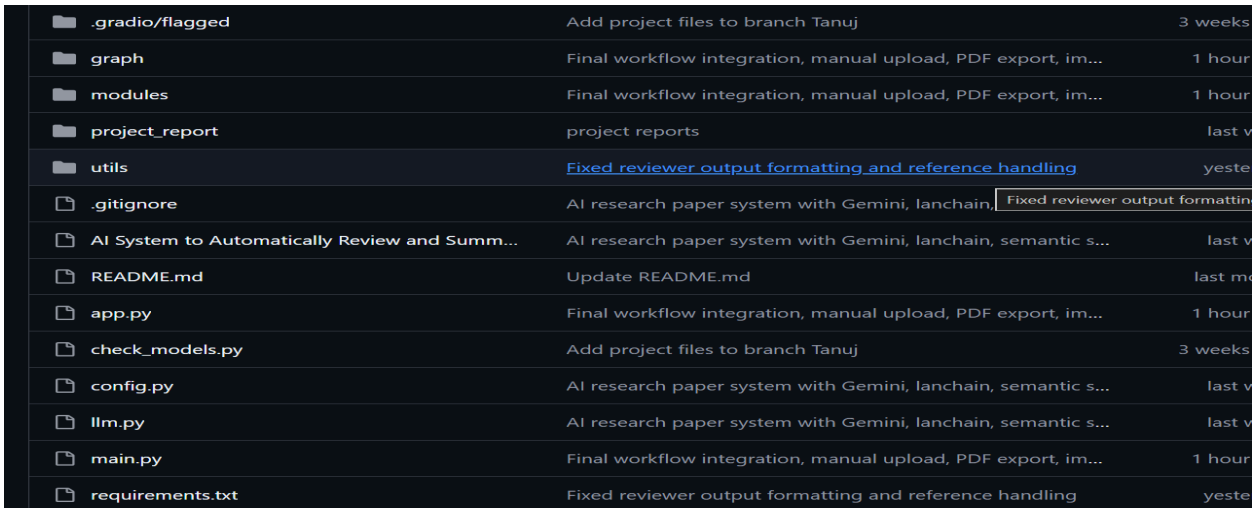
summary_20260211_123430	11-02-2026 12:34	Text Document	9 KB
summary_20260211_124419	11-02-2026 12:44	Microsoft Edge PDF ...	6 KB
summary_20260211_124419	11-02-2026 12:44	Text Document	6 KB
summary_20260211_125355	11-02-2026 12:53	Microsoft Edge PDF ...	6 KB
summary_20260211_125355	11-02-2026 12:53	Text Document	6 KB
summary_20260211_130041	11-02-2026 13:00	Microsoft Edge PDF ...	6 KB
summary_20260211_130041	11-02-2026 13:00	Text Document	6 KB
summary_20260211_151425	11-02-2026 15:14	Microsoft Edge PDF ...	6 KB
summary_20260211_151425	11-02-2026 15:14	Text Document	5 KB
summary_20260211_152725	11-02-2026 15:27	Microsoft Edge PDF ...	5 KB
summary_20260211_152725	11-02-2026 15:27	Text Document	5 KB
summary_20260212_124559	12-02-2026 12:45	Microsoft Edge PDF ...	5 KB
summary_20260212_124559	12-02-2026 12:45	Text Document	5 KB
summary_20260212_142849	12-02-2026 14:28	Microsoft Edge PDF ...	6 KB
summary_20260212_142849	12-02-2026 14:28	Text Document	5 KB
summary_20260212_150914	12-02-2026 15:09	Microsoft Edge PDF ...	5 KB
summary_20260212_150914	12-02-2026 15:09	Text Document	5 KB
summary_20260212_151045	12-02-2026 15:10	Microsoft Edge PDF ...	5 KB
summary_20260212_151045	12-02-2026 15:10	Text Document	5 KB
summary_20260212_151749	12-02-2026 15:17	Microsoft Edge PDF ...	5 KB
summary_20260212_151749	12-02-2026 15:17	Text Document	5 KB
summary_20260212_153722_078792	12-02-2026 15:37	Microsoft Edge PDF ...	3 KB
summary_20260212_153722_078792	12-02-2026 15:37	Text Document	2 KB



Features Implemented

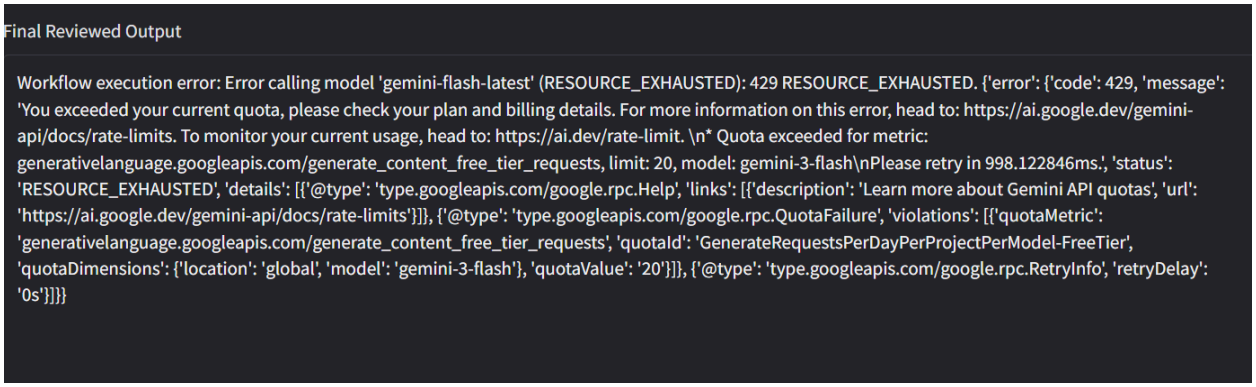
- Automatic & manual modes
- Section-wise comparison
- AI-based structured summary

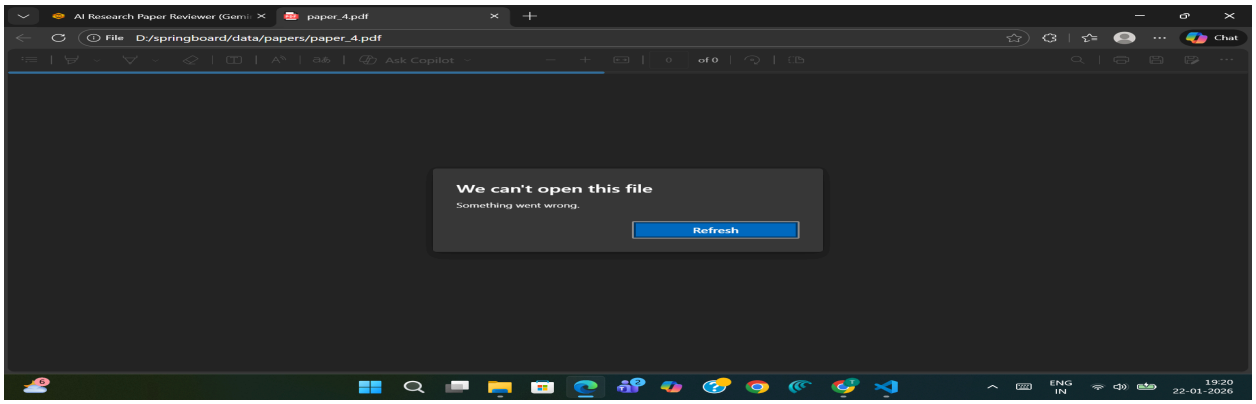
- Academic formatting
- APA reference formatting
- TXT + PDF export
- GitHub-ready project structure



Challenges Faced

- Gemini API quota limits
- Handling invalid PDF URLs
- Token length constraints
- Mixed LLM output formats (list/dict/string)
- Ensuring minimum two-paper comparison





Solutions:

- Output normalization
- Error handling
- Text truncation
- Structured validation

Conclusion

This project successfully demonstrates how Generative AI can automate systematic literature review.

The system is:

- Modular
- Scalable
- Practical for academic use
- Suitable for industry research automation

It aligns strongly with Infosys Springboard objectives by demonstrating:

- AI/LLM integration
- API handling
- Modular architecture
- End-to-end system development

Future Enhancements

- Support for more than 3 papers
- LangGraph-based visual workflow
- Word document export
- Cloud deployment
- Database integration